Arcade Mania – Tesztdokumentáció

A projekt tesztelése során több különböző tesztelési módszert és keretrendszert alkalmaztunk annak érdekében, hogy a rendszer különböző részei megfelelően működjenek. A tesztelés során ellenőriztük a frontend komponensek működését, az API kommunikációt, valamint a backend logikát és az adatbázis műveleteket is. A frontend alkalmazás React és Vite környezetben készült, míg a backend ASP.NET Core Web API technológiával lett megvalósítva.

A frontend tesztek futtatása Node.js környezetben történt, Vitest keretrendszer segítségével. A backend tesztek során Microsoft MSTest teszteket használtunk. A funkcionális tesztekhez pedig Selenium WebDriver-t használtunk, Python környezetben. A backend teszteket Bódi Szabolcs, a frontend teszteket Oravecz Ádám József készítette.

A tesztdokumentumban nem szerepel, minden teszt, ugyanis a dokumentum méretkorlátja ezt nem tette lehetővé, így a tesztdokumentum csak 5 db tesztet tartalmaz a teljesség igény nélkül.

React Komponens teszt – Login.test.jsx

Az alábbi teszt a bejelentkezési felület működését ellenőrzi. Sikeres bejelentkezés esetén megjelenik a sikeres visszajelzés, hiányzó adatok esetén megjelenik a megfelelő hibaüzenet.

A teszt a következőképpen néz ki:

```
import { test, expect, afterEach } from 'vitest'
import '../setupTests'
import { render, fireEvent, waitFor, cleanup } from '@testing-library/react'
import { MemoryRouter } from 'react-router-dom'
import Login from '../pages/Login'
test('successful login shows success message', async () => {
  const mockLogin = async (u) => ({ id: 1, username: u })

  const { getByText, container } = render(
    <MemoryRouter>
      <Login loginFn={mockLogin} initialValues={{ username: 'alice', password: 'secret' }} />
    </MemoryRouter>
  )

  fireEvent.submit(container.querySelector('form'))

  await waitFor(() => {
    expect(getByText('Succes')).toBeInTheDocument()
  })
})

afterEach(() => cleanup())

test('shows error when credentials missing', async () => {
  const { getByText, container } = render(
    <MemoryRouter>
      <Login />
    </MemoryRouter>
  )

  fireEvent.submit(container.querySelector('form'))

  await waitFor(() => {
    expect(getByText(/Felhasználónév és jelszó megadása kötelező./)).toBeInTheDocument()
  })
})
```

src/__test__/Login.test.jsx:

Login > successful login shows success message

A teszt sikeres. A bejelentkezési komponens megfelelően működik: a form elküldése után a rendszer sikeres bejelentkezés esetén megjeleníti a visszajelzést a felhasználó számára.

Login > shows error when credentials missing

A teszt sikeres. A rendszer ellenőrzi, hogy a felhasználónév és jelszó meg van-e adva. Ha a felhasználó üresen küldi el az űrlapot, a komponens megfelelő hibaüzenetet jelenít meg.

React Egységteszt – userService.test.js

A teszt során – több másik eset mellett – azt ellenőriztük, hogy a függvény helyesen dolgozza-e fel a szerver válaszát, valamint hibát dob-e hibás adat esetén.

```
import { describe, test, expect, afterEach } from 'vitest'
import { strict as assert } from 'assert'
import * as userService from '../services/userService'

describe('userService', () => {
  const originalFetch = globalThis.fetch

  afterEach(() => {
    globalThis.fetch = originalFetch
  })

  test('getPublicUser returns result on valid response', async () => {
    globalThis.fetch = async () => ({
      status: 200,
      ok: true,
      text: async () => JSON.stringify({ message: '', result: { id: 1, name: 'Alice' } })
    })

    const res = await userService.getPublicUser(1)
    assert.deepEqual(res, { id: 1, name: 'Alice' })
  })

  test('invalid JSON response throws', async () => {
    globalThis.fetch = async () => ({
      status: 200,
      ok: true,
      text: async () => 'not-json'
    })

    await expect(userService.getPublicUser(3)).rejects.toThrow('Érvénytelen JSON válasz érkezett a szervertől.')
  })
})
```

src/__test__/userService.test.js:

userService > getPublicUser returns result on valid response

A teszt sikeres. A getPublicUser függvény megfelelően dolgozza fel a szervertől érkező válasz adatokat, és a felhasználói objektumot adja vissza a frontend számára.

userService > invalid JSON response throws

A teszt sikeres. A szolgáltatás megfelelően kezeli a hibás szerverválaszt: ha a visszakapott adat nem érvényes JSON formátumú, a függvény hibát dob.

Selenium WebDriver – funkcionális teszt – FrontEndSeleniumTest.py

A Selenium lehetővé teszi a böngésző vezérlését, így a teszt valós felhasználói műveleteket szimulál. A teszt Chrome böngészőben futott.

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
import time

# Setup
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
wait = WebDriverWait(driver, 10)

try:
    # 1. Oldal megnyitása
    driver.get("http://localhost:5173") # Ellenőrizd a portot a Bun termináljában!

    # 2. login folyamat
    # Az első input a "Username" placeholderrel
    user_field = wait.until(EC.presence_of_element_located(
        (By.XPATH, "//input[@placeholder='Username']")))

    # A második input a "Password" placeholderrel
    pass_field = driver.find_element(By.XPATH, "//input[@placeholder='Password']")

    # A submit gomb
    login_btn = driver.find_element(By.XPATH, "//button[@type='submit']")

    # Adatok kitöltése
    # az adatbázisban már alapvetően létező adatokat használjuk
    time.sleep(1)
    user_field.send_keys("admin")
    time.sleep(1)
    pass_field.send_keys("admin")
    time.sleep(1)
    login_btn.click()
    time.sleep(2)  # Várunk egy kicsit, hogy a login folyamat befejeződjön

    # 3. PLAY GAME
    # Itt várunk, amíg a React lerendereli a gombot a login után
    # Feltételezem, hogy a gomb szövege "Play Game"
    play_btn = wait.until(EC.element_to_be_clickable(
        (By.XPATH, "//button[contains(text(), 'Play Game')]")))
    play_btn.click()
    print("Belépve és Play megnyomva.")

    time.sleep(5)
```

```python
    time.sleep(5)

    # 4. BACK GOMB
    # Keressük a szövege alapján (ha nincs ID)
    back_btn = wait.until(EC.element_to_be_clickable(
        (By.CLASS_NAME, "back")))  # Vagy használj egyedi osztályt vagy szöveg alapján
    back_btn.click()

    time.sleep(1)

    # 5. LOGOUT (Dropdown kezeléssel)
    print("Kijelentkezés megkezdése...")

    # Először megnyitjuk a dropdown menüt (a gomb, ami kinyitja)
    # A képed alapján 'dropdown-btn' volt az osztálya
    dropdown_trigger = wait.until(EC.element_to_be_clickable((By.CLASS_NAME, "dropdown-btn")))
    dropdown_trigger.click()

    # Várunk egy picit, amíg a React "kiteriti" a menüt (animáció miatt)
    time.sleep(0.5)

    # Most már kattintható a Logout link
    logout_link = wait.until(EC.element_to_be_clickable((By.LINK_TEXT, "Logout")))
    logout_link.click()

    print("Teszt sikeresen lefutott!")

except Exception as e:
    print(f"Hiba történt: {e}")
    driver.save_screenshot("error.png")

finally:
    driver.quit()
```

/FrontEndSeleniumTest.py:

Selenium > A teszt sikeresen lefutott!

A teszt sikeres. Az elvártak szerint bejelentkezik, kiválasztja egy egyik játékot, visszalép és kilép a profilból.

Backend API teszt – bejelentkezési folyamat teszt, integrációs teszt – UsersController_LoginTest.cs

```dotnet
[TestMethod]
0 references
public async Task Login_Success_Returns200_WithToken_AndScores()
{

    var db = Guid.NewGuid().ToString();

    using var ctx = TestHelpers.CreateInMemoryContext(db);

    var games = TestHelpers.SeedGamesReturnEntities(ctx, "Snake", "Tetris");

    var cfg = TestHelpers.CreateTestConfig();

    var pwKey = cfg["Crypto:PasswordKey"]!;

    var userId = Guid.NewGuid();

    ctx.Users.Add(new User
    {
        Id = userId,
        UserName = "Elek",
        PasswordHash = EncryptProvider.AESEncrypt("pass123", pwKey),
        Role = "User"
    });

    // score rekordok (login visszaadja Join-olva)
    ctx.UserHighScores.Add(new UserHighScore { UserId = userId, GameId = games[0].Id, HighScore = 11u });

    ctx.UserHighScores.Add(new UserHighScore { UserId = userId, GameId = games[1].Id, HighScore = 22u });

    await ctx.SaveChangesAsync();

    var controller = TestHelpers.CreateUsersController(ctx, cfg);

    var action = await controller.Login(new UserLoginDto { Name = "Elek", Password = "pass123" });

    var obj = action as ObjectResult;

    Assert.IsNotNull(obj);

    Assert.AreEqual(200, obj!.StatusCode);

    // response anon object: { message, token, result }
    var token = TestHelpers.GetAnonymousProp<string>(obj.Value!, "token");

    Assert.IsFalse(string.IsNullOrWhiteSpace(token), "Sikeres login esetén token kötelező.");


    var result = TestHelpers.GetAnonymousProp<object>(obj.Value!, "result");

    Assert.IsNotNull(result);

    var scores = TestHelpers.GetAnonymousProp<object>(result!, "Scores");

    Assert.IsNotNull(scores, "Login resultnek tartalmaznia kell Scores-t.");
}
```

TestProject/UsersController_LoginTests.cs

UsersController -> Login_Success_Returns200_WithToken_AndScores

A teszt sikeres. A bejelentkezés sikeres és a 200-as státuszkódot kapjuk vissza.

Backend API teszt – felhasználó regisztráció teszt, integrációs teszt – UsersController_RegisterTest.cs

```
[TestMethod]
0 references
public async Task Register_HappyPath_CreatesUser_AndCreatesScoresForAllGames()
{

    // Arrange
    var dbName = Guid.NewGuid().ToString();

    using var ctx = TestHelpers.CreateInMemoryContext(dbName);

    TestHelpers.SeedGames(ctx, "Snake", "Tetris", "Pacman");

    var config = TestHelpers.CreateTestConfig();

    var controller = TestHelpers.CreateUsersController(ctx, config);

    var dto = new UserRegisterDto
    {
        Name = "TesztElek",
        Password = "pass123"
    };

    // Act
    var action = await controller.Register(dto);

    // Assert (HTTP)
    var obj = action as ObjectResult;

    Assert.IsNotNull(obj, "Register-nek ObjectResult-ot kell visszaadnia.");

    Assert.AreEqual(201, obj!.StatusCode, "Sikeres regisztrációnál 201-et várunk.");

    // Assert (DB)
    var user = await ctx.Users.SingleOrDefaultAsync(u => u.UserName == "TesztElek");

    Assert.IsNotNull(user, "A usernek létre kellett jönnie.");

    Assert.AreEqual("User", user!.Role, "Register után default role 'User' kell legyen.");

    Assert.IsFalse(string.IsNullOrWhiteSpace(user.PasswordHash), "PasswordHash nem lehet üres.");

    var gamesCount = await ctx.Games.CountAsync();

    var scoresCount = await ctx.UserHighScores.CountAsync(s => s.UserId == user.Id);

    Assert.AreEqual(gamesCount, scoresCount, "Minden Game-hez kell egy UserHighScore rekord 0-val.");

    Assert.IsTrue(await ctx.UserHighScores.AllAsync(s => s.HighScore == 0), "Kezdő highscore mindenhol 0 kell legyen.");
}
```

TestProject/UsersController_RegisterTests.cs

UsersController -> Register_HappyPath_CreatesUser_AndCreatesScoresForAllGames

A teszt sikeres. A teszt után a felhasználó valóban, sikeresen létrejött.

Megjegyzés:

A további tesztek a frontend és a backend vizsgarészekben találhatóak meg, futtatásra kész állapotban.